AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING
i-CREDIT HOURS ENGINEERING PROGRAMS
Computer Engineering and Software Systems
Computer and Artificial Intelligence Engineering

CSE 485 / CSE 488: Deep Learning

Final Project: *From Pixels to Language*

Instructors: Dr. Mahmoud Khalil, Eng. Seif Yasser

1 Summary

This is an end-to-end multimodal deep learning project. Your team of **3 to 5 students** will build a system that (1) **extracts spatial features** from images by training a CNN encoder for classification, (2) **reconstructs degraded visual data** by colorizing grayscale images, and (3) **generates natural language text** by producing captions that describe an image. The same visual encoder is the backbone of all three tasks, so what you learn in one task carries into the next.

Project Marks: 20 marks (Bylaw 2023 - CSE488) | 25 marks (Bylaw 2018 - CSE485)

2 Objectives

The goal of this project is to master the full deep learning pipeline, end to end: exploratory data analysis (EDA), data preprocessing and augmentation, training a CNN from scratch, transfer learning, model explainability, generative vision (image-to-image), and sequence modeling (image-to-text). Along the way you will practise the engineering side of deep learning: writing modular and reproducible code, tracking experiments, tuning hyperparameters, evaluating models properly, and benchmarking your results against others' work.

3 Dataset

3.1 Default Dataset: CUB-200-2011 with Captions

The default dataset is the **Caltech-UCSD Birds-200-2011 (CUB-200-2011)** dataset together with its natural language descriptions (Reed et al., 2016). It is available on Hugging Face as two datasets, one per split: `Multimodal-Fatima/CUB_train` and `Multimodal-Fatima/CUB_test` (original homepage: `https://www.vision.caltech.edu/datasets/cub_200_2011/`).

It contains photos of **200 bird species**, mostly from North America, and every image comes with **ten human-written descriptions**:

- **Total images:** 11,788 (about 1.1 GB)
- **Training set:** 5,994 images
- **Test set:** 5,794 images

You can load it with:

```
from datasets import load_dataset
train = load_dataset("Multimodal-Fatima/CUB_train", split="train")
test  = load_dataset("Multimodal-Fatima/CUB_test",  split="test")
```

Data Structure

Each record has the following fields, which cover all three tasks of the project:

Field	Description	Used for
`image`	The bird photograph.	All tasks
`label`	The species as a class label (200 classes), e.g. *Black footed Albatross*.	Classification
`description`	One text holding ten captions of the bird, separated by new lines (see the note below).	Captioning
`file_name`	The original image file name, e.g. `Black_Footed_Albatross_0007_796138.jpg`.	Reference
`id`	A row identifier.	Reference

Example record (first three of its ten captions shown):

```
label:     Black footed Albatross
file_name: Black_Footed_Albatross_0007_796138.jpg
description:
  this bird is brown with a lighter brown crest.
  aquatic large bird with long hooked bill, white face, and brown body.
  bird has brown body feathers, brown breast feathers, and brown beak
  ...
```

The description column needs preprocessing

The `description` field is **one text that holds all ten captions**, separated by new lines (`\n`). It is **not** ready to use as it is. You must preprocess it first. Analyze the captions as part of your EDA (Task 1.1), and explain every preprocessing step you choose in your report.

The original CUB-200-2011 release also provides bounding boxes, segmentation masks, 15 part locations, and 312 binary attributes per image. They are not part of this version and are not required, but you may download them from the homepage if they help your work.

Data Samples

Figure 1 shows samples from the dataset with their labels and captions.



Hooded Merganser
"a crazy looking bird with a black and white head and neck with brown textured wings."

Vermilion Flycatcher
"a red bellied bird with a red breast and a red crown, with a black superciliary."

White necked Raven
"a black bird with white on its nape."

Swainson Warbler
"the bird has a white eyebrow, white breast, and brown plumage atop its crown, wings, and back"

Figure 1: Samples from CUB-200-2011 with their label and one of their ten captions.

Building the Colorization Data

The dataset does not ship grayscale/color pairs, and it does not need to: you can build them yourself. Convert every image to grayscale and keep the original; each image then gives you an input (grayscale) and its target (the original colored version), as in Figure 2.



Figure 2: Grayscale inputs and their original colored targets for the colorization task.

3.2 Using a Different Dataset

You may use a different dataset instead of CUB-200-2011. Alternative datasets are approved as long as they suit all three problems of this project:

1. **Image classification:** each image must have a discrete class label.
2. **Image colorization:** the images must be of good enough quality for image-to-image work. As with CUB, you can build the data yourself by converting the images to grayscale, which gives each image a grayscale version and its original colored version.
3. **Image captioning:** each image must have natural language text (captions) describing it.

Compute constraint

Whatever dataset you choose must be lightweight enough (**under 2 GB**) to be trained on free **Google Colab** or **Kaggle** (T4) GPUs, or on your own local machine.

4 General Requirements

- **Teams:** 3–5 students per team.
- **GitHub from day one:** all work must be kept in a GitHub repository from the first day of the project.
- **Individual accounts:** each member must commit from their own GitHub account. Commits must represent meaningful, incremental work. Uploading everything at the deadline is **unacceptable**.
- **Individual grades:** project grades are individual. Your discussion performance, technical understanding, and verifiable GitHub contributions determine your final grade.

5 Technical Requirements

5.1 Tools

- **Language and framework:** Python and PyTorch.
- **Visualization:** Matplotlib (other plotting libraries may be used alongside it).
- **Experiment tracking:** MLflow, Weights & Biases, or any other experiment tracking tool.

5.2 Code Design and Project Structure

Your code must be **modular** and follow **SOLID principles** and suitable **design patterns**. Data loading, models, training, evaluation, and visualization should live in separate modules, so that, for example, you can swap one encoder for another without rewriting your training loop. A suggested layout (adapt it to your needs, do not over-engineer it):

```
project/
|-- README.md
|-- requirements.txt
|-- configs/            # hyperparameters and run settings
|-- notebooks/          # experiments only (not part of the final code)
|-- src/
|   |-- data/           # datasets, transforms, augmentation
|   |-- models/         # encoder, colorization model, caption decoder
|   |-- training/       # training loops
|   |-- evaluation/     # metrics, benchmarking
|   |-- visualization/  # plots, Grad-CAM
|   |-- cli.py          # command line entry point
|-- outputs/            # figures, logs (large files not committed)
```

5.3 Command Line Interface

The project must have a **Command Line Interface (CLI)** for both **training** and **inference** on every task. A GUI is optional. For example:

```
python -m src.cli train   --task classification --config configs/cls.yaml
python -m src.cli predict --task caption --image path/to/bird.jpg
```

5.4 Notebooks Policy

Notebooks are for experiments only

Jupyter/Colab notebooks are allowed **only for trying and testing ideas**. The final code must be **pure Python** (`.py` modules run through the CLI). You may include notebooks that show your trials and experiments in your submission, but they are **not** part of the main code and cannot replace it.

5.5 Transfer Learning vs. Training from Scratch

You are allowed, and encouraged, to use **transfer learning** from pre-trained models to achieve high accuracy. However, you **must** also train your own CNN encoder **completely from scratch** to establish a baseline, and compare the two.

5.6 Explainability (Grad-CAM)

You **must** implement **Grad-CAM** (Gradient-weighted Class Activation Mapping) to visualize what your models are seeing: where their attention is focused and how the network behaves on specific predictions, both correct and wrong.

5.7 Experiment Tracking

Use your experiment tracking tool to log every run: hyperparameters, metrics, training and validation curves, and model checkpoints or their references. You should be able to show any run and explain how it differs from the others.

5.8 Hyperparameter Tuning

Hyperparameter tuning is **required in Milestone 1** and **optional in Milestone 2**. You do not need to tune every hyperparameter, only a subset of them. When you tune:

- Use both **Grid Search** and **Random Search**.
- **Optuna** (or a similar tool) is optional.
- Compare the tuning methods against each other **at least once** (e.g., best score reached, number of trials, time taken).

5.9 Visualizations

Use visualizations throughout the project, including:

- **Data:** sample images, class distribution, image statistics, augmentation examples.
- **Training:** training vs. validation loss and accuracy (or other metric) curves.
- **Model comparison:** from-scratch vs. transferred models, architecture experiments, tuning results, and the final selected model.
- **Explainability:** Grad-CAM heatmaps over images.
- **Outputs:** colorized images next to their grayscale input and ground truth; generated captions next to reference captions.

5.10 Evaluation Metrics

Evaluate each model with **more than one metric**. For every metric you use, explain **what it is**, **what it measures**, and **what your score means**. The following are examples only; you may choose others:

- **Classification:** accuracy, precision, recall, F1-score, confusion matrix.
- **Colorization:** PSNR, MAE.
- **Captioning:** BLEU, ROUGE-L.

5.11 Benchmarking

Benchmark your models against **open-source models** or **published results** on the dataset you use, where they exist. You can either:

- run open-source models yourselves on your data and evaluate them with your metrics, or
- compare directly against results reported by others.

Your results do **not** need to be higher. The goal is to learn how to benchmark against others' work: make sure the comparison is fair (same split, same metrics) and explain where the differences come from.

6 Milestones

The project is split into two milestones.

Milestone	Tasks	Deadline
Milestone 1	EDA, classification and encoder training	Week 7 (1 Nov)
Milestone 2	Colorization and captioning	Week 14 (18 Dec)

Milestone 1 is **not graded separately**, but its work **must be submitted through the LMS** by its deadline.

Project discussions will start on **19 Dec**.

6.1 Milestone 1

6.1.1 Task 1.1: Data Understanding & EDA

Analyze the dataset: class distribution, image sizes and quality, caption statistics (length, vocabulary, typos), and anything unusual (imbalance, duplicates, corrupted files, noisy captions). Plot your findings, clean and preprocess the data (including splitting and cleaning the `description` column), and design a suitable augmentation strategy.

6.1.2 Task 1.2: Image Classification & Encoder Training

Train a CNN encoder **from scratch** for image classification, then apply **transfer learning** with pre-trained models and compare the results. Tune hyperparameters, track your experiments, and use **Grad-CAM** to visualize and explain your models' predictions.

6.2 Milestone 2

6.2.1 Task 2.1: Image Colorization

Colorize grayscale images using your trained encoder as the backbone of an image-to-image model (for example, an encoder–decoder or U-Net). Build the grayscale/color pairs as described in Section 3.

6.2.2 Task 2.2: Image-to-Text Generation (Captioning)

Freeze the visual encoder, map its latent space into a **decoder** (Transformer, RNN, LSTM, or GRU), and generate captions **autoregressively**.

7 Project Documentation and Report

Your report must document **anything and everything you have done**, including what did not work. It must be submitted as a **PDF**; writing it in LaTeX is recommended but optional. It should cover:

1. **Introduction:** the problem, the dataset you chose and why.
2. **Data:** EDA findings with plots, cleaning, preprocessing, and augmentation, with justifications.
3. **Models:** every architecture you tried, for every task, with diagrams or descriptions.
4. **Experiments:** all experiments with their settings, including baseline vs. tuned models and from-scratch vs. transfer learning.
5. **Hyperparameter tuning:** search spaces, methods used, and the comparison between them.
6. **Experiment tracking:** screenshots or exports from your tracking tool.
7. **Evaluation:** the metrics you used, what each one measures, and an interpretation of your scores.
8. **Explainability:** Grad-CAM results with your analysis of correct and wrong predictions.
9. **Benchmarking:** your results vs. open-source models or published results, and how the comparison was done.
10. **Final selection:** the final model for each task and why you selected it.
11. **Failures and limitations:** what did not work, why you think so, and what you would try next.
12. **Team contributions:** who did what.
13. **References:** papers, repositories, and models you used.

The `README.md` must explain how to set up the environment and give the exact commands to run training and inference.

8 Final Deliverables

Put all of the following in a single archive named `TeamXX.zip` (where `XX` is your team number; a `.rar` archive is also accepted) and upload it to the **LMS**:

- A Google Drive link containing all trained models.
- The GitHub source code link, with the full commit history.
- `README.md` with environment setup and exact run commands.

- `requirements.txt` or equivalent.
- The project report (PDF), as described in Section 7.
- A video demo showing the final end-to-end system.

9 Grading Criteria

Grades are **individual**. The breakdown is:

Criterion	Weight
Discussion	35%
Report	10%
Data Understanding, EDA, Cleaning, Preprocessing, Augmentation	10%
Code Quality and Reproducibility	5%
Inference/Training Interface	5%
Task 1: Classification / Encoder	10%
Task 2: Image Colorization	10%
Task 3: Image-to-Text Generation	10%
Benchmarking	5%
Total	**100%**

9.1 Discussion (35%)

The discussion will be held **offline** (in person). Each student is discussed individually. You are graded on your technical understanding of the whole project, your ability to explain and justify the decisions made, and your verifiable contributions on GitHub. Every member must understand the entire project, not only their own part.

9.2 Report (10%)

Graded on completeness, clarity, and honesty, following Section 7: everything you did must be documented, with justified decisions, clear figures, and well-interpreted results.

9.3 Data Understanding, EDA, Cleaning, Preprocessing, and Augmentation (10%)

Graded on how well you understand your data (Task 1.1): meaningful analysis and plots, justified cleaning and preprocessing, an augmentation strategy suited to the data, and how these findings informed your modeling choices.

9.4 Code Quality and Reproducibility (5%)

Graded on modular and clean code, a clear project structure, pure Python final code (notebooks for experiments only), and whether someone else can reproduce your results from the `README.md` and `requirements.txt`.

9.5 Inference/Training Interface (5%)

Graded on a working CLI that can train and run inference for every task, with clear arguments and usage instructions.

9.6 Task 1: Classification / Encoder (10%)

Graded on the classification and encoder task as a whole: how complete and correct your work is, the quality of your results, and how well you understand and justify what you did.

9.7 Task 2: Image Colorization (10%)

Graded on the colorization task as a whole: how complete and correct your work is, the quality of your results, and how well you understand and justify what you did.

9.8 Task 3: Image-to-Text Generation (10%)

Graded on the captioning task as a whole: how complete and correct your work is, the quality of your results, and how well you understand and justify what you did.

Applies to all three tasks

Experiment tracking and visualizations are graded **inside each task**, and each must be done **at least once** in the project. Hyperparameter tuning is **required in Milestone 1** (graded in Task 1) and **optional in Milestone 2**.

9.9 Benchmarking (5%)

Graded on comparing your models against open-source models or published results: a fair comparison (same split, same metrics), and a clear explanation of the differences. Scoring lower than others is acceptable.

10 AI Usage Guidelines

AI should accelerate our work, not replace our engineering and research judgment, you steer the AI, and DO NOT let it steer you, you are the one in control and you choose how should it behave.

- **Think first.** Before asking AI to solve a problem, break it down yourself and form an initial approach: what needs to be done, why, and how you would approach it.
- **Drive the outcome.** Refine the plan based on your own judgment and AI's input. You will end up with an implementable plan. You remain responsible for the final approach and decisions.
- **Define the criteria for each step.** Before asking AI to implement a step, clearly define what you expect from it and how, including interfaces, inputs, outputs, constraints, data structures, and expected behavior.
- **Understand what AI generates.** You MUST understand and review generated code. Do not accept code simply because it compiles or passes tests.
- **Use AI for research.** AI research tools can accelerate investigation, but always verify important claims against reliable sources.

Think → Plan → Challenge with AI → Refine → Define → Build Incrementally → Review → Validate.